АЛГОРИТМЫН ҮНДЭС · PYTHON · ХАЙЛТ 1/2

Linear search

Шугаман хайлт: бичих, мөрдөх, хувилбарууд, тестлэх



Түвшин: Хэрэгжүүлэх | **Жишээ:** Python 3 | **Урьдчилсан мэдлэг:** list, for давталт, функц

Энэ хичээлийг уншсаны дараа та:

- Linear search-ийг **өөрөө бичиж**, мөр бүр юу хийдгийг тайлбарлана.
- Кодоо цаасан дээр **мөрдөж** (trace), алхам бүрийг хүснэгтээр гаргана.
- Linear search-ийн **3 түгээмэл алдааг** таньж засна.
- Сүүлийн тохиолдол, бүх байрлал, нөхцөлөөр хайх гэх мэт **хувилбаруудыг** бичнэ.
- Кодоо **тестээр** шалгаж, linear search-ийг хэзээ хэрэглэх, хэзээ хэрэглэхгүйг мэднэ.

Агуулга

01 Хайлтын бодлого
02 Санаа ба код
03 Гараар мөрдөх
04 Түгээмэл алдаа
05 Хэр хурдан бэ?
06 Хувилбарууд
07 Тестлэх
08 Хэзээ хэрэглэх вэ?
09 Дасгал
10 Хураангуй

> **Нэг өгүүлбэрээр**
> **Linear search** (шугаман хайлт): жагсаалтыг эхнээс нь нэг бүрчлэн хар, олдмогц зогс; бүгдийг хараад олдоогүй бол — байхгүй.

01 Хайлтын бодлого

Хайлт гэдэг нь цуглуулгаас тодорхой нэг утгыг олох бодлого. Бичихээсээ өмнө функц юу авч, юу буцаахыг тохиролцъё — үүнийг функцийн **гэрээ** (contract) гэдэг:

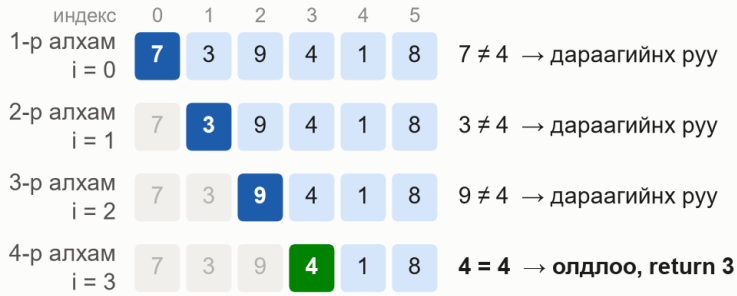
	Тайлбар	Жишээ
Оролт	`items` — жагсаалт; `target` — хайж буй утга	`items = [7, 3, 9, 4]`
Олдвол	`target`-ийн **индекс** (0-ээс эхэлж тоолно)	`search(items, 9) → 2`
Олдохгүй бол	`-1`	`search(items, 5) → -1`

Яагаад −1 вэ?
Жинхэнэ индекс 0, 1, 2, … гэж үргэлж 0 ба түүнээс их байдаг. Тиймээс −1 хэзээ ч жинхэнэ индекстэй андуурагдахгүй — «олдсонгүй» гэдгийг илэрхийлэхэд тохиромжтой. Python-ий `str.find()` яг ингэж −1 буцаадаг бол `list.index()` олдохгүй үед `ValueError` алдаа шиддэг.

Анхаар: Python-д items[-1] бол сүүлийн элемент
Буцсан утгыг шалгалгүй `items[idx]` гэж хэрэглэвэл олдоогүй үед ч алдаа гарахгүй, чимээгүйхэн **сүүлийн** элементийг авна. Үргэлж `if idx != -1:` гэж шалгаарай.

02 Санаа ба код

Зүйрлэл: 6 шүүгээний алинд нь түлхүүрээ хийснээ мартжээ. Шүүгээг зүүнээс нь нэг нэгээр онгойлгоно: олдвол тэр даруй зогсоно. Бүгдийг онгойлгоод олдохгүй бол — түлхүүр шүүгээнд **байхгүй** гэсэн үг.



Зураг 1. [7, 3, 9, 4, 1, 8] жагсаалтаас 4-ийг хайх нь. Хар хөх — одоо шалгаж буй, саарал — аль хэдийн шалгасан, цайвар хөх — хараахан шалгаагүй. 4 алхамд олдлоо.

Алхам алхмаар бичье

Алхам 1. Индекс бүрээр давтана: `for i in range(len(items)):`

Алхам 2. Тухайн элементийг хайж буй утгатай харьцуулна: `if items[i] == target:`

Алхам 3. Таарвал индексийг **тэр даруй** буцаана: `return i` — функц шууд дуусна, үлдсэнийг харахгүй.

Алхам 4. Давталт бүтэн дуусвал (бүгдийг харсан, олдоогүй) `return -1` — давталтын **гадна**!

```python
def linear_search(items, target):
    for i in range(len(items)):     # 1. индекс бүрээр давтана
        if items[i] == target:      # 2. харьцуулна
            return i                # 3. олдлоо → шууд гарна
    return -1                       # 4. бүгдийг харсан, олдсонгүй

nums = [7, 3, 9, 4, 1, 8]
print(linear_search(nums, 4))       # 3
print(linear_search(nums, 5))       # -1
```

Python-д илүү түгээмэл бичиглэл бол `enumerate` — индекс ба утгыг зэрэг өгдөг. Үр дүн нь яг ижил:

```python
def linear_search(items, target):
    for i, x in enumerate(items):   # i — индекс, x — утга
        if x == target:
            return i
    return -1
```

> **Яагаад индекс буцаадаг вэ?**
> Утга нь бидэнд аль хэдийн байгаа — target-ийг өөрөө өгсөн. Харин **хаана** байгаа нь шинэ мэдээлэл: индексээр нь тэр элементийг өөрчлөх (`items[i] = ...`), устгах (`del items[i]`), хөршийг нь харах (`items[i + 1]`) боломжтой.

03 Гараар мөрдөх (trace)

Код бичсэний дараах хамгийн сайн шалгалт бол цаасан дээр **мөрдөх**: хүснэгт зурж, давталтын эргэлт бүрт хувьсагчдын утгыг бичнэ. Хоёр тохиолдлыг мөрдье.

Тохиолдол 1: `linear_search([7, 3, 9, 4, 1, 8], 4)` — олдоно

i	items[i]	items[i] == 4 ?	Юу болох вэ
0	7	False	дараагийн эргэлт рүү
1	3	False	дараагийн эргэлт рүү
2	9	False	дараагийн эргэлт рүү
3	4	**True**	`return 3` — функц дуусна

4 харьцуулалт хийлээ. 1, 8 гэсэн үлдсэн элементийг **огт хараагүй** — `return` функцийг шууд дуусгадаг.

Тохиолдол 2: `linear_search([7, 3, 9, 4], 5)` — байхгүй



Зураг 2. [7, 3, 9, 4] жагсаалтаас 5-ыг хайх нь. Бүх элементийг шалгаж дуусаад л −1 буцаана.

i	items[i]	items[i] == 5 ?	Юу болох вэ
0	7	False	дараагийн эргэлт рүү
1	3	False	дараагийн эргэлт рүү
2	9	False	дараагийн эргэлт рүү
3	4	False	давталт дууслаа
—			`return -1` — давталтын дараах мөр

Зөвлөгөө: компьютерээр мөрдүүлэх

Давталт дотор нэг мөр `print` нэмбэл хүснэгтийг Python өөрөө хэвлэнэ. Код буруу ажиллаж байвал хамгийн түрүүнд үүнийг хий:

```
for i in range(len(items)):
    print(f"i={i}  items[i]={items[i]}  тэнцүү юу: {items[i] == target}")
    ...
```

04 Түгээмэл алдаа

Linear search ердөө 4 мөр ч эхлэгчид доорх 3 алдааг байнга гаргадаг. Бүгдийг нь бодитоор ажиллуулж шалгав.

Алдаа 1 · return -1-ийг давталт дотор бичих

БУРУУ

```
for i in range(len(items)):
    if items[i] == target:
        return i
    else:
        return -1
```

ЗӨВ

```
for i in range(len(items)):
    if items[i] == target:
        return i
return -1
```

Юу болох вэ: `linear_search([7, 3, 9, 4], 9)` → **−1**, гэтэл 9 индекс 2-т байна!

Яагаад: Эхний элемент 7 ≠ 9 болмогц else салаа ажиллаж функц дуусчихна — бусад элементийг огт харахгүй. «Олдсонгүй» гэж зөвхөн **бүгдийг** харсны дараа хэлж болно.

Алдаа 2 · Индексийн оронд утга буцаах

БУРУУ

```
for x in items:
    if x == target:
        return x
return -1
```

ЗӨВ

```
for i, x in enumerate(items):
    if x == target:
        return i
return -1
```

Юу болох вэ: `linear_search([7, 3, 9, 4], 9)` → **9** (индекс 2 биш).

Яагаад: `x` нь утга, индекс биш. Буцаасан 9 нь бидний аль хэдийн мэдэх target өөрөө — ямар ч шинэ мэдээлэл өгөхгүй. Индекс хэрэгтэй бол `enumerate` эсвэл `range(len(items))`.

Алдаа 3 · Сүүлийн элементийг алгасах

БУРУУ

```
for i in range(len(items) - 1):
    ...
```

ЗӨВ

```
for i in range(len(items)):
    ...
```

Юу болох вэ: `linear_search([4, 6, 8], 8)` → **−1**.

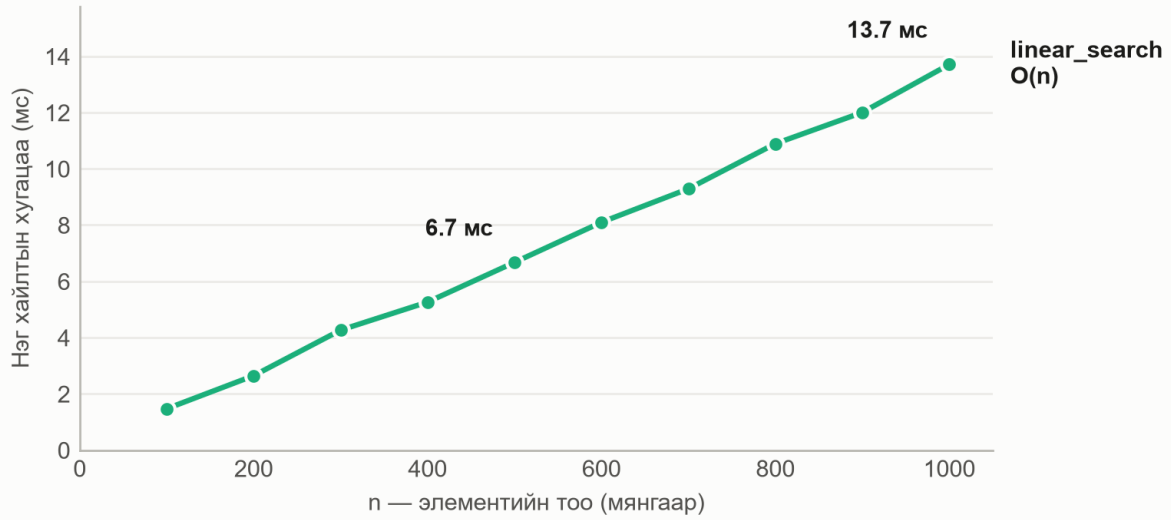
Яагаад: `range(n)` нь 0, 1, …, n−1 гэж аль хэдийн **сүүлийн индекс хүртэл** гүйдэг. `- 1` нэмбэл сүүлийн элемент (индекс 2) шалгагдахгүй. Ийм «нэгээр зөрөх» (off-by-one) алдаа маш түгээмэл.

05 Хэр хурдан бэ?

Хурдыг секундээр биш, **харьцуулалтын тоогоор** хэмжье. n элементтэй жагсаалтад:

Тохиолдол	Хэзээ	Харьцуулалт	Big O
Хамгийн сайн	target эхний байранд	1	O(1)
Дундаж	target санамсаргүй байранд	≈ n / 2	O(n)
Хамгийн муу	target сүүлд эсвэл **байхгүй**	n	**O(n)**
Санах ой	хэдхэн хувьсагч (i)	—	O(1)

Big O нь ихэвчлэн **хамгийн муу** тохиолдлыг хэлдэг тул linear search бол **O(n)**: жагсаалт 2 дахин томорвол ажил 2 дахин нэмэгдэнэ. Бодитоор хэмжиж шалгая — 100 000 … 1 000 000 элементтэй жагсаалтаас **байхгүй** утга хайлгав (хамгийн муу тохиолдол):



Зураг 3. linear_search-ийн нэг хайлтын хугацаа. Энэ PDF-ийг үүсгэх үед бодитоор хэмжсэн (хэд хэдэн давталтын хамгийн бага утга). Таны компьютер дээр тоо нь өөр гарах ч хэлбэр нь ижил — шулуун шугам.

500 000 элементэд **6.7 мс**, 1 000 000 элементэд **13.7 мс** — бараг яг **2.1 дахин**. Энэ бол O(n)-ийн «n 2 дахин → ажил 2 дахин» гэдгийн бодит нотолгоо.

06 Хувилбарууд

«Давт → шалга → буцаа» гэсэн загварыг бага зэрэг өөрчлөөд олон бодлого бодож болно. Өөрчлөгдсөн хэсгийг тайлбараар тэмдэглэв.

а) Сүүлийн тохиолдлыг олох

```python
def linear_search_last(items, target):
    for i in range(len(items) - 1, -1, -1):   # ← арааc нь урагш: n-1, n-2, …, 0
        if items[i] == target:
            return i
    return -1

linear_search_last([5, 2, 5, 7, 5, 1], 5)     # 4
```

`range(start, stop, step)`: `stop` өөрөө орохгүй тул 0 хүртэл явахын тулд `-1` гэж бичнэ.

б) Бүх байрлалыг олох

```python
def find_all(items, target):
    result = []                    # ← олдсон бүрийг цуглуулна
    for i, x in enumerate(items):
        if x == target:
            result.append(i)       # ← буцаахгүй, үргэлжлүүлнэ
    return result                  # олдоогүй бол хоосон жагсаалт []

find_all([5, 2, 5, 7, 5, 1], 5)     # [0, 2, 4]
```

в) Нөхцөлөөр хайх

Тэнцүү байхыг биш, **ямар нэг нөхцөл** хангахыг шалгаж болно — энэ бол linear search-ийн том давуу тал (binary search үүнийг ерөнхий тохиолдолд чадахгүй).

```python
def first_failing(scores):
    for i, score in enumerate(scores):
        if score < 60:             # ← == биш, дурын нөхцөл
            return i
    return -1

first_failing([88, 75, 54, 91, 42])   # 2
```

г) Объектын жагсаалтаас хайх

```
students = [
    {"name": "Бат", "score": 88},
    {"name": "Сараа", "score": 95},
    {"name": "Тэмүүлэн", "score": 72},
]

def find_student(students, name):
    for s in students:
        if s["name"] == name:       # ← зөвхөн нэрээр нь харьцуулна
            return s                # индекс биш, объектыг өөрийг нь буцаалаа
    return None                     # олдоогүй: объект буцаадаг үед None нь тохиромжтой

find_student(students, "Сараа")     # {'name': 'Сараа', 'score': 95}
```

д) Файлаас хайх

Linear search-д индексээр «үсрэх» шаардлагагүй, зөвхөн дараалаар нь уншихад л хангалттай. Тиймээс бүхэлд нь санах ойд ачаалахгүйгээр том файлаас ч хайж чадна:

```
def find_line(path, word):
    with open(path, encoding="utf-8") as f:
        for no, line in enumerate(f, 1):    # мөрийг нэг нэгээр уншина, 1-ээс дугаарлана
            if word in line:
                return no
    return -1
```

Python-д бэлэн байгаа linear search-үүд

Код	Буцаах утга	Цаг
`x in items`	True / False	O(n)
`items.index(x)`	эхний индекс; олдохгүй бол `ValueError`	O(n)
`items.count(x)`	хэдэн удаа байгаа (үргэлж бүгдийг харна)	O(n)

> **Санамж**
> Эдгээр нь дотроо яг манай давталтыг хийдэг. Тиймээс давталт дотор `x in list` бичвэл нийтдээ **O(n²)** болно (Big O хичээлийн «нуугдмал давталт»).

07 Тестлэх

«Ажиллаж байх шиг байна» гэж орхиж болохгүй — 4-р бүлгийн алдаанууд зарим жишээн дээр зөв хариу өгдөг! Хоёр төрлийн тест бичье.

1. Захын тохиолдлууд (edge cases)

Алдаа ихэвчлэн хил дээр нуугддаг: хоосон, ганц элемент, эхний ба сүүлийн байр.

```
def test_edges():
    assert linear_search([], 5) == -1          # хоосон жагсаалт
    assert linear_search([5], 5) == 0          # ганц элемент, байгаа
    assert linear_search([5], 3) == -1         # ганц элемент, байхгүй
    assert linear_search([4, 6, 8], 4) == 0    # эхний элемент
    assert linear_search([4, 6, 8], 8) == 2    # сүүлийн элемент  ← алдаа 1, 3 энд баригдана
    assert linear_search([4, 6, 8], 7) == -1   # байхгүй
    assert linear_search([2, 7, 2], 2) == 0    # давхардсан → ЭХНИЙХ
    print("Захын тохиолдлууд: OK")
```

`assert` нь нөхцөл худал бол `AssertionError` шиднэ, үнэн бол юу ч хийхгүй.

2. Санамсаргүй тест: бэлэн функцтэй харьцуулах

Python-ий `in` ба `index`-ээр «зөв хариу» гаргаж, мянга мянган санамсаргүй жагсаалт дээр өөрийн функцтэйгээ харьцуулна:

```
import random

def reference(items, target):           # бэлэн функцээр «зөв хариу»
    return items.index(target) if target in items else -1

def test_random(trials=10_000):
    for _ in range(trials):
        items = [random.randint(0, 9) for _ in range(random.randint(0, 15))]
        target = random.randint(-2, 11)  # заримдаа байхгүй утга
        assert linear_search(items, target) == reference(items, target), \
            (items, target)              # бүтэлгүйтвэл жишээг хэвлэнэ
    print(trials, "санамсаргүй тест: OK")
```

Энэ PDF-ийг үүсгэх үед зөв хувилбарыг **10 000** санамсаргүй тестээр шалгахад бүгд амжилттай давлаа. Харин 4-р бүлгийн алдаатай хувилбаруудыг ижил тестэд оруулахад:

Хувилбар	1 000 тестээс бүтэлгүйтсэн	Анхны алдаа	Алдааны төрөл
Алдаа 1: `else: return -1`	**316**	1-р тест	буруу хариу
Алдаа 2: `return x`	**293**	1-р тест	буруу хариу
Алдаа 3: `range(len(items) - 1)`	**29**	24-р тест	буруу хариу
Зөв `linear_search`	**0**	—	—

Анхаар: 3-р алдаа 1 000 тестээс ердөө **29**-д илэрсэн — target **зөвхөн сүүлийн** байранд байх үед л буруу ажилладаг. Ийм ховор алдааг санамсаргүй тест олохгүй өнгөрч болох тул захын тохиолдлын тест («сүүлийн элемент») заавал хэрэгтэй.

> **Дүгнэлт**
> Захын тохиолдлын тест + бэлэн функцтэй харьцуулах санамсаргүй тест = хэрэгжүүлэлтдээ итгэх хамгийн хялбар арга. Алдаа бүр эхний 24 тестийн дотор илэрсэн.

08 Хэзээ хэрэглэх вэ?

Давуу тал	Сул тал
Ямар ч жагсаалт дээр ажиллана — **эрэмбэлэх шаардлагагүй**	Том жагсаалтад удаан: сая элементэд сая хүртэл харьцуулалт
Дурын нөхцөлөөр хайж чадна (эхний тэгш тоо, 60-аас бага оноо…)	Олон удаа хайхад ажил нь давтагдсаар байна: k хайлт → k · n
Файл, генератор гэх мэт зөвхөн дараалаар уншдаг өгөгдөлд тохирно	
4 мөр — алдаа гаргахад хэцүү, уншихад амархан	

Шийдвэрийн жагсаалт

1. Жагсаалт **жижиг** (хэдэн арав, хэдэн зуу) эсвэл **ганцхан удаа** хайх уу? → **Linear search**. Энгийн, хангалттай хурдан.
2. **Дурын нөхцөлөөр** хайх уу? → **Linear search**.
3. Өгөгдөл **файл, сүлжээ** зэргээс дараалаар ирж байна уу? → **Linear search**.
4. Зөвхөн «байгаа юу?» гэж **олон удаа** асуух уу? → `set` эсвэл `dict`: дунджаар O(1) (Big O хичээлийн 5-р бүлэг).
5. **Эрэмбэлэгдсэн** том жагсаалтаас олон удаа хайх уу? → **Binary search** — дараагийн хичээл.

> **Дараагийн алхам: Binary search**
> Утасны дэвтрийг эхний хуудаснаас нь нэг бүрчлэн уншдаггүй — дундаас нь нээгээд хагасыг нь хаядаг. Жагсаалт **эрэмбэлэгдсэн** бол яг ингэж сая элементээс **20** алхамд олж болно. Энэ бол «Хайлт 2/2: Binary search» хичээлийн сэдэв.

09 Дасгал

Эхлээд өөрөө бодож, кодоо 7-р бүлгийн аргаар тестлээрэй. Хариуг дараагийн хуудаснаас шалга.

Дасгал 1 · Тоолох

`count_occurrences(items, target)` функц бич: target хэдэн удаа байгааг буцаана. `count()` хэрэглэхгүй. Цагийн төвөгшил нь хэд вэ?

```
count_occurrences([1, 2, 2, 5, 2], 2)    # 3
```

Дасгал 2 · Гараар мөрдөх

`linear_search([5, 8, 2, 9, 2], 2)` ба `linear_search([5, 8, 2, 9, 2], 7)` хоёрыг 3-р бүлгийн хүснэгтээр мөрд. Тус бүр хэдэн харьцуулалт хийж, юу буцаах вэ?

Дасгал 3 · Алдааг ол

Доорх код `linear_search([4, 6, 8], 8)`-д юу буцаах вэ? Яагаад? Хэрхэн засах вэ?

```
def linear_search(items, target):
    for i in range(len(items)):
        if items[i] == target:
            return i
        else:
            return -1
```

Дасгал 4 · Хоёр дахь тохиолдол

`second_occurrence(items, target)`: target-ийн **хоёр дахь** удаа гарсан индексийг буцаа, тийм байхгүй бол −1.

```
second_occurrence([5, 2, 5, 7, 5], 5)    # 2
second_occurrence([5, 2], 5)             # -1
```

Дасгал 5 · Нөхцөлөөр хайх

`first_greater(items, x)`: x-ээс **их** эхний элементийн индексийг буцаа (жагсаалт эрэмбэгүй байж болно).

```
first_greater([3, 9, 4, 12, 7], 5)    # 1   (9 > 5)
first_greater([1, 2], 5)              # -1
```

Дасгалын хариу

Бүх хариуг энэ PDF-ийг үүсгэх үед жишээн дээр нь ажиллуулж шалгасан.

1. Тоолох — O(n)

```
def count_occurrences(items, target):
    count = 0
    for x in items:
        if x == target:
            count += 1              # буцаахгүй — тоолood үргэлжлүүлнэ
    return count                    # O(n): бүгдийг заавал харна
```

2. Гараар мөрдөх

target = 2: **3** харьцуулалт, `2` буцаана. target = 7: **5** харьцуулалт (бүгдийг), `-1` буцаана.

i	items[i]	== 2 ?	== 7 ?
0	5	False	False
1	8	False	False
2	2	**True** → `return 2`	False
3	9	—	False
4	2	—	False
—			`return -1`

3. Алдааг ол — −1 буцаана

Эхний элемент 4 ≠ 8 тул else салаа ажиллаж шууд `return -1` хийнэ — 6, 8-ыг огт харахгүй. Засвар: else-ийг устгаад `return -1`-ийг давталтын **гадна** (for-той ижил догол) бичнэ. Энэ бол 4-р бүлгийн 1-р алдаа.

4. Хоёр дахь тохиолдол — O(n)

```
def second_occurrence(items, target):
    seen = 0                        # хэд дэх удаагаа олж байгааг тоолно
    for i, x in enumerate(items):
        if x == target:
            seen += 1
            if seen == 2:
                return i
    return -1
```

5. Нөхцөлөөр хайх — O(n)

```
def first_greater(items, x):
    for i, v in enumerate(items):
        if v > x:                   # «== target»-ийн оронд нөхцөл
            return i
    return -1
```

10 Хураангуй

Загвар — цээжлэх биш, ойлгож бич

■ Linear search · O(n)

```
def linear_search(items, target):
    for i in range(len(items)):
        if items[i] == target:
            return i
    return -1   # давталтын ГАДНА
```

Ижил санаа, enumerate-тэй

```
def linear_search(items, target):
    for i, x in enumerate(items):
        if x == target:
            return i
    return -1
```

Linear search бичихдээ шалгах 4 зүйл

1. **Бүх** индексийг хамарч байна уу? `range(len(items))` — `- 1`-гүй.
2. Олдмогц **индексийг** (утгыг биш) `return` хийж байна уу?
3. `return -1` давталтын **гадна** байна уу?
4. Захын тохиолдлуудаар (хоосон, 1 элемент, эхний, сүүлийн, байхгүй, давхардсан) **тестэлсэн** үү?

Санаж үлдэх зүйлс

- Олдохгүй бол `-1`; хэрэглэхээсээ өмнө `if idx != -1` гэж шалга.
- «Олдсонгүй» гэж зөвхөн **бүгдийг** харсны дараа хэлнэ.
- Цаг: хамгийн муу үед **O(n)**; санах ой: **O(1)**. n 2 дахин → ажил 2 дахин.
- `in`, `index()`, `count()` бол бэлэн linear search — давталт дотор хэрэглэхдээ болгоомжил.
- Эрэмбэгүй, жижиг, нөхцөлөөр хайх, дарааллаар ирэх өгөгдөлд — linear search.
- Том, эрэмбэлэгдсэн жагсаалтаас олон удаа хайх бол — **binary search** (Хайлт 2/2).